Jev typed decisions: six patterns, seven clients and a replay project

One interface, different applications

The sources introduce six decision patterns, seven community repositories and an application to market snapshots. TypeSafe's quickstart (https://docs.typesafe.ai/introduction/quickstart) documents the actual interface and links to the console (https://console.typesafe.ai) for access. Choice, Score and Noul are typed questions. Routing is a use of a choice, not a fourth output type. Confidence is not a guaranteed correct decision.

Current setup

The official Python quickstart specifies Python 3.10+ and:

```
python -m pip install typesafe-sdk
```

Use the documented `TypeSafeClient`, `system_one`, `Choice`, `Score` and `Noul` examples. Keep `TYPESAFE_API_KEY` outside source control. The hosted endpoint is `https://api.typesafe.ai/v1/systemone`. Check the current model alias, access and billing before making calls. Source guides retain September pricing and access reports; this combined guide does not promote them to current guarantees.

Six reusable patterns

Use typed questions for model routing, intent triage, RAG passage scoring, agent continue/stop decisions and policy classification. Tool-call risk classification can be one signal, but deterministic permissions and human decision boundaries must remain outside the model. Compare each pattern with a keyword/rule or simple ranking baseline before adopting a network dependency.

The seven community clients

These are separate projects and should be evaluated on their own README, license and maintenance:

- jev-ultrafast (https://github.com/browser-use/jev-ultrafast): browser action selection.
- fast-jev-compaction (https://github.com/tamaratran/fast-jev-compaction): context-selection plugin.
- pg-jev (https://github.com/realZachi/pg-jev): database classification functions; consider which row data leaves the database.
- tax-doc-classifier (https://github.com/kyotofin/tax-doc-classifier): document-type classification.
- jev-curate (https://github.com/AkashPriyadarshii/jev-curate): content curation.
- jev-chat-jarvis (https://github.com/jev-chat/jev-chat-jarvis): assistant/client integration.
- awesome-jev (https://github.com/yibie/awesome-jev): a resource list, not a required runtime client. Open client code does not provide self-hostable Jev weights. Repository benchmark claims and star counts in the earlier guide are dated source observations, not measurements performed here.

The market example is a research project

Replay timestamped snapshots through a mock adapter, then optionally Jev. Measure full wall-clock latency, timeouts, stale feeds and calibration on held-out examples. Keep inventory limits, cancellation, rate limits and halt behavior deterministic. Simulate fees, spread and slippage; favorable midpoint marks are not realized fills. This does not establish profitable trading or a guaranteed HFT latency budget.

```
Build a replay-only typed-decision benchmark with mock and optional Jev
adapters. Define fixtures, deterministic limits, timeout behavior and a
no-model baseline. Report latency distributions and calibration with dated
configuration. Do not add live orders or claim profitability.
```

Original slides, detailed six-pattern alternatives, repository analysis and the newer replay guide remain downloadable. The main topic connects their shared mechanism while keeping the trading claims bounded.

Original sources

- https://www.instagram.com/p/Ddm2r7KkiXU/?img_index=1
- https://www.instagram.com/p/DdtkBHBiXBj/
- https://www.instagram.com/reel/DeAr43JPbqv/